

Registro semana 6 mobile

Nome: Arthur Saraiva de França 3B.

Objetivo da atividade

O objetivo desta atividade foi compreender como funcionam operações assíncronas no Android, utilizando duas abordagens: **AsyncTask** e **Coroutines**, comparando suas diferenças na prática.

Contexto

Foi desenvolvido um exemplo de aplicativo de notas, onde é necessário salvar uma nota em segundo plano, para evitar travamentos na interface do usuário.

Parte 1 – Implementação com AsyncTask

A classe AsyncTask foi utilizada para executar uma tarefa em segundo plano.

Código exemplo:

```
public class SaveNoteTask extends AsyncTask<String, Void, String> {
```

```
    @Override
    protected void onPreExecute() {
        super.onPreExecute();
        System.out.println("Salvando nota...");
    }
```

```
    @Override
    protected String doInBackground(String... notes) {
        String note = notes[0];

        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        return note;
    }
```

```
@Override
protected void onPostExecute(String result) {
    super.onPostExecute(result);
    System.out.println("Nota salva: " + result);
}
}
```

Explicação simples

- `onPreExecute()` → executa antes (ex: mostrar carregando)
 - `doInBackground()` → executa em segundo plano
 - `onPostExecute()` → executa depois (atualiza a tela)
-

Parte 2 – Implementação com Coroutines

Foi utilizada uma abordagem moderna com Kotlin e Coroutines.

Código exemplo:

```
fun saveNoteWithCoroutines(note: String) {
    GlobalScope.launch(Dispatchers.IO) {

        Thread.sleep(2000)

        withContext(Dispatchers.Main) {
            println("Nota salva: $note")
        }
    }
}
```

Explicação simples

- `GlobalScope.launch` → inicia a tarefa
 - `Dispatchers.IO` → roda em segundo plano
 - `withContext(Dispatchers.Main)` → volta pra tela principal
-

Parte 3 – Comparação das abordagens

AsyncTask

- Mais antiga

- Mais difícil de entender
- Pode causar erros e vazamentos de memória
- Já foi descontinuada (deprecated)

Coroutines

- Mais moderna
- Código mais simples e organizado
- Mais fácil de manter
- Mais segura

Qual usar em um projeto real?

Eu utilizaria **Coroutines**, pois são mais modernas, mais fáceis de entender e evitam problemas comuns do AsyncTask, como vazamentos de memória e código complexo.

Resultado obtido

Foi possível implementar as duas formas de execução assíncrona e entender suas diferenças na prática.

Conclusão

A atividade ajudou a entender como executar tarefas em segundo plano no Android. Foi possível perceber que as Coroutines são uma solução mais eficiente e recomendada atualmente.